

Coding Framework Verdict: agent- skills vs mattpocock/skills

Aibar Yerzhuman

August 31, 2026

CONTENTS

Coding Framework Verdict: agent-skills vs mattpocock/skills	
Executive verdict	
Your measured task profile	
Framework head-to-head	
agent-skills — all 24 skills scored	
mattpocock/skills — all 37 skills scored	
Your recommended daily lineup	
Actions (pending your go-ahead)	

Coding Framework Verdict: agent-skills vs mattpocock/skills

Prepared for: Aibar Yerzhuman · 2026-08-31 **Method:** task profile mined from 50 real Claude Code sessions (2026-05-26 → 2026-08-26, Octarin history); every skill in both repos read in full (24 in addyosmani/agent-skills @ #505, 37 in mattpocock/skills @ v1.2.3/HEAD 2026-08-24) and scored against that profile.

Executive verdict

Keep agent-skills as the execution spine. Keep mattpocock as the specialist bench. Do not switch frameworks — they lose in opposite places.

- **agent-skills wins the loop.** Its spec → plan → build chain is the most coherent, artifact-driven execution system of the two (capability maps, `tasks/todo.md`, per-slice verification), and your one-approval-gate fork of it is already installed and validated live (2/2 tests, per-task commits, no re-review). mattpocock's equivalent spine (grill → to-spec → to-tickets → implement) is a near-total duplicate of it, mediated through an issue tracker with setup ceremony and one-decision-per-session pacing — precisely the process weight you rejected.
- **mattpocock wins the bench.** Its standalone skills are the most concrete in your environment and cost almost nothing always-on (11 model-invokable descriptions ≈ 650 tokens, already installed). `diagnosing-bugs` (7/10) is the best debugging skill you own and targets your 18% firefighting bucket; `wizard`, `research`, and `writing-for-agents` (6/10 each) cover ops walkthroughs, sourced exploration, and harness authoring — none of which agent-skills addresses at all.
- **Both frameworks share one blind spot: your biggest bucket.** You are ~32% exploration/orientation. agent-skills serves it with 0 of 24 skills (its router has no “understand this repo” branch; `interview-me` explicitly refuses information requests). mattpocock serves it with exactly one thin skill (`research`). Your orientation file + doc index + gstack `investigate` remain the real coverage here — no framework replaces them.

Net changes recommended (3 promotions, 0 removals): promote `api-and-interface-design` and `observability-and-instrumentation` from agent-skills into the harness (the only two of its remaining 19 that fill genuine gaps — queue idempotency and incident observability for the live outreach system), and invoke `diagnosing-bugs` by name during production incidents.

Your measured task profile

Bucket	Share	What it looks like in your sessions
Exploration / orientation	~32%	Repo architecture mapping, feasibility research, API investigation (WhatsApp Business API, scraper market audits)
Feature delivery	~26%	Next.js 16 landing pages (bilingual EN/RU), Node/TS outreach pipelines, data tooling (PDF extraction, exam analytics)
Fix / diagnose	~18%	Production firefighting: queue collapse, batch recovery, migration blockers — urgent, on a live email system
Review / gatekeeping	~8%	Task-closure re-reviews, pricing/docs audits, design-variant audits
Ops	~8%	Vercel deploys, env config, macOS tooling
Docs / writing	~4%	Agreements, plans, PDFs

Scoring rubric per skill: **DAILY score = 0.5·task-fit + 0.2·concreteness + 0.3·marginal value over your existing stack** (duplicates score low regardless of quality). Existing stack counted: your installed 5-skill core, karpathy-guidelines, gstack (investigate/review/ship/browse), sight loop (Playwright+odiff), accessibility-audit, Octarin.

Framework head-to-head

Dimension	agent-skills (Osmani)	mattpocock/skills
Shape	24 skills, one lifecycle (define→plan→build→verify→review→ship), 8 commands, 4 subagent personas, hooks, shared references, CI evals	37 skills (25 shipped in plugin), documented main flow + on-ramps, prose-only orchestration (no hooks/agents)
Execution loop	Winner. Artifact-driven chain already installed as /spec /plan /build; single approval gate (your fork); validated end-to-end	Duplicate spine, tracker-mediated, setup ceremony (<code>/setup-matt-pocock-skills</code> , triage labels) — the process weight you rejected
Exploration coverage (your 32%)	0 of 24 skills — router has no orientation branch	1 of 37 (<code>research</code> , 6/10) — thin but real
Firefighting coverage (your 18%)	<code>debugging-and-error-recovery</code> (5/10 — duplicated by your stack)	diagnosing-bugs (7/10 — best debugging skill in your environment)
Standalone skill concreteness	High (rationalization tables, red flags)	Highest (bundled scripts/templates, completion criteria — e.g. wizard ships a bash template)
Always-on cost as installed	~350 tokens (5 descriptions, selective copy)	~650 tokens (11 model-invokable of 25; 14 are command-only)
Maintenance	Very active (~500 PRs; HEAD 2026-08-21)	Very active (changesets releases;

Dimension	agent-skills (Osmani)	mattpocock/skills
		HEAD 2026-08-24; installed 1.2.3 = current release, no functional drift)
Ceremony risk	Present in defaults (4-gate spec, mandatory doubt cycles) — neutralized by your fork	Present in the spine (multi-session wayfinding, tracker state) — avoided by not adopting the spine
Best use for you	The loop	The bench

agent-skills — all 24 skills scored

Score format: **DAILY** (fit/concreteness/marginal), each 0–10.

Skill	What it does	When to use it	Score
incremental-implementation	Delivers changes as thin vertical slices: test → verify → commit per slice; scope-discipline rules	Any multi-file feature or change; it is the engine behind <code>/build</code>	7 (7/7/6)
spec-driven-development	Writes a structured spec (objective, commands, structure, style, testing, boundaries; capability map for multi-module work) before any code	Starting a feature or project where requirements are vague; behind <code>/spec</code>	6 (6/7/6)
planning-and-task-breakdown	Breaks a spec into dependency-ordered tasks with acceptance criteria (<code>tasks/plan.md</code> + <code>todo.md</code>)	Work feels too large to start, or before <code>/build auto</code> ; behind <code>/plan</code>	6 (6/7/6)
interview-me	One-question-at-a-time interview (with a guess attached to each) until ~95% confidence on intent	An ask is underspecified — “build X” without for-whom or why-now; won’t fire for pure information requests	6 (5/8/5)
test-driven-development	Red-green-refactor: failing test before code, Prove-It pattern for bug fixes	Implementing logic or fixing a bug in pipelines/data tooling; wired into <code>/build</code>	6 (6/8/4)
api-and-interface-design	Contract-first API/interface design, incl. a deep idempotency-key, retry, and DLQ-retention protocol	Designing or changing queue and API contracts — e.g. outreach send paths, batch claim logic	6 (5/8/6)
observability-and-instrumentation	Starts from on-call questions, then structured logs, RED metrics, symptom-based alerts	Adding logging/metrics/alerts to a live system so the next incident is a query, not archaeology	6 (4/8/7)

Skill	What it does	When to use it	Score
debugging-and-error-recovery	Stop-the-line triage: reproduce → localize → reduce → root-cause → guard with a regression test	A build or test breaks mid-implementation and the cause isn't obvious	5 (6/7/2)
frontend-ui-engineering	Production UI checklist: responsive breakpoints, WCAG, avoiding recognizable "AI aesthetic" patterns	Building UI when no dedicated design/a11y skill is loaded	5 (6/7/2)
deprecation-and-migration	Safe removal and expand/contract schema migrations: dual-write, backfill, contract	Schema or behavior changes on a live database where in-place renames would break traffic	5 (3/7/6)
source-driven-development	Verifies framework usage against freshly fetched official docs, with cited URLs	Writing against fast-moving APIs (Next 16, React 19) where training data may be stale	5 (5/7/3)
browser-testing-with-devtools	Chrome DevTools MCP verification loop plus untrusted-page-content security rules	Browser-verifying a change when no other browser tooling is set up	4 (4/7/2)
git-workflow-and-versioning	Atomic commits, worktrees, semver, changelog discipline, scope-discipline sections	Repo-hygiene questions: how to slice commits, when to branch, how to version	4 (5/6/2)
ci-cd-and-automation	GitHub Actions quality-gate pipelines where no gate can be skipped	Setting up or overhauling CI on a repo	4 (2/8/4)
performance-optimization	Measure → fix → re-measure with a keep-or-revert ledger ("Neutral" is a revert)	Performance work outside the web/CWV domain	4 (3/8/2)
security-and-hardening	STRIDE threat modeling, OWASP, SSRF, supply chain, LLM and privacy hardening	A deliberate security pass on a feature that crosses trust boundaries	4 (4/8/2)

Skill	What it does	When to use it	Score
code-review-and-quality	Five-axis diff review with severity labels and change sizing	A structured review pass on a diff before merge	3 (3/7/1)
code-simplification	Behavior-preserving clarity refactors that respect project idiom	Cleanup passes after a feature lands	3 (3/7/1)
context-engineering	Guidance on rules files, selective context loading, surfacing confusion	Designing how an agent's context/rules are laid out in a repo	3 (3/5/2)
documentation-and-adrs	ADRs, READMEs, changelogs, why-comments	Formal decision records or docs passes	3 (2/6/2)
idea-refine	Divergent→convergent ideation ending in a one-pager with a “Not Doing” list	Shaping a fuzzy product idea before it's ready for a spec	3 (3/6/2)
doubt-driven-development	Adversarial fresh-context review of every non-trivial decision, with a cross-model check	High-stakes, irreversible decisions where a deliberate second pass is worth the ceremony	3 (2/7/1)
shipping-and-launch	Pre-launch checklist, canary percentages, rollback plans	Staged production launches beyond a plain Vercel deploy	3 (3/7/1)
using-agent-skills	Meta-router mapping tasks to the framework's skills + core behaviors	Running the full 24-skill framework install; irrelevant in a selective install	2 (2/5/1)

mattpocock/skills — all 37 skills scored

(in-progress and misc skills are repo-only — not shipped in the installed plugin)

Skill	What it does	When to use it	Score
diagnosing-bugs	Refuses to theorize until a tight one-command repro loop exists (10 ranked loop-construction methods), then reproduce → minimize → ranked hypotheses → instrument → regression-test → cleanup	Any production incident or non-obvious bug — queue collapse, batch failures, “it worked yesterday”	7 (8/9/5)
wizard	Generates an interactive bash script (from a bundled template) that walks a human through credential, dashboard, and migration steps, writing values to <code>.env</code> and gh secrets	Third-party service onboarding: API signups, env/secrets wiring, Vercel/dashboard setup	6 (5/9/7)
research	Background investigation against primary sources only; leaves a cited markdown file in the repo	Feasibility, API, or market research where sourced claims matter	6 (8/5/4)
writing-for-agents	Reference on writing skills and AGENTS.md files: context vs cognitive load, information hierarchy, leading words, no-op pruning	Authoring or editing agent skills, CLAUDE.md/AGENTS.md, or harness docs	6 (5/6/7)
prototype	Throwaway code answering one design question — logic questions become a single shareable HTML state-machine player; UI questions become variants behind a URL param	Validating a state model or design direction before touching real code (e.g. a queue lifecycle)	5 (5/8/3)

Skill	What it does	When to use it	Score
resolving-merge-conflicts	Resolves each conflict hunk by tracing both sides' intent through commits/PRs/issues; never aborts mid-operation	An in-progress merge or rebase with non-trivial conflicts	5 (3/7/6)
retro (<i>in-progress</i>)	Session retrospective proposing environment fixes across 7 categories (navigation, checks, no-ops, tool economy, information access)	After a session that felt inefficient, to turn friction into environment changes	5 (4/7/5)
setup-ts-deep-modules (<i>in-progress</i>)	Wires dependency-cruiser to enforce entry-point-only package imports and proves the rules bite	Hardening module boundaries in a growing TS codebase; once per repo	5 (2/9/6)
code-review	Two-axis diff review (12-smell standards baseline + spec fidelity) in parallel subagents	A structured pre-merge review pass on a diff	4 (4/8/2)
codebase-design	Deep-module design vocabulary (depth, seams, leverage, locality) plus a deletion test	Reasoning about module boundaries and where abstraction earns its keep	4 (3/6/5)
domain-modeling	Maintains a CONTEXT.md glossary + ADRs and challenges fuzzy terms inline	Keeping shared vocabulary honest on a project where terms drift	4 (3/6/3)
tdd	TDD reference: pre-agreed test seams, anti-patterns (tautological tests, horizontal slicing), red-before-green	Test-driving logic; same seat as the installed test-driven-development skill	4 (4/6/1)
to-spec	Synthesizes the current conversation into a user-story spec published to the issue tracker, no interview	Turning a discussion into a spec artifact; same seat as <code>/spec</code>	4 (4/7/1)

Skill	What it does	When to use it	Score
to-tickets	Breaks a spec into tracer-bullet vertical slices with blocking edges; expand–contract sequencing for wide refactors	Decomposing a spec into ordered tickets; same seat as <code>/plan</code>	4 (4/8/1)
grilling	Interview primitive: numbered question rounds over a design-tree frontier, each question carrying a recommended answer	Stress-testing a design or requirement set; same seat as interview-me	4 (4/7/2)
handoff	Compacts the conversation into a redacted handoff doc (with suggested next skills) in a temp dir	Passing work to another session, person, or model	4 (4/5/3)
to-questionnaire	Interviews you about “the send,” then writes a most-important-first questionnaire for someone else to answer	Collecting requirements or decisions from a stakeholder asynchronously	4 (2/7/6)
wayfinder	Charts a large foggy effort as a map issue plus decision tickets with fog-of-war; one decision resolved per session	Multi-week ambiguous efforts too big to spec in one sitting	4 (2/8/4)
improve-codebase-architecture	Scans hot spots for module-deepening opportunities and renders a Tailwind+Mermaid HTML report	Spare-cycle architecture upkeep between feature pushes	4 (2/8/5)
setup-pre-commit	Installs husky + lint-staged + Prettier, verified by committing through the new hook	New-repo setup; fires once per project	4 (2/9/5)
claude-handoff (in-progress)	Hands the compacted summary straight into a	Spawning a background continuation of the current	4 (4/6/4)

Skill	What it does	When to use it	Score
	<code>c laude --bg</code> background agent	work	
ask-matt	Prose router mapping every skill into flows (main flow, on-ramps, phase boundaries, context-hygiene rules)	Learning or adopting the full mattpocock framework	3 (2/6/2)
implement	8-line wrapper: implement from tickets, use tdd, typecheck, code-review, commit	Executing tracker tickets; same seat as <code>/build</code>	3 (5/3/1)
triage	Issue/PR state machine (5 canonical roles): verify claims, write agent briefs, maintain an out-of-scope KB	Maintaining a repo with an inbound issue queue	3 (1/8/3)
setup-matt-pocock-skills	One-time scaffold of issue-tracker config, triage labels, and domain-doc layout	Adopting the full mattpocock flow in a repo	3 (1/8/2)
teach	Stateful multi-session tutoring workspace (MISSION.md, HTML lessons, learning records)	Structured multi-session learning of a topic	3 (1/8/4)
wait-what	Re-explains the last message in Simplified Technical English using the project glossary	An explanation didn't land and you want it replayed plainly	3 (3/4/3)
implement-spec (<i>in-progress</i>)	Parallel implementer subagents in git worktrees over a ticket frontier, merger subagents, one PR	Parallel implementation of an approved spec; same territory as <code>/build auto</code> + Conductor	3 (3/6/2)
loop-me (<i>in-progress</i>)	Grills recurring life/work "loops" into delegatable workflow specs	Systematizing a personal workflow you repeat weekly	3 (2/6/4)

Skill	What it does	When to use it	Score
writing-beats / writing- fragments / writing-shape (<i>in-progress</i>)	Long-form writing pipeline: mine fragments (explore) → shape a structure (exploit) → assemble beat-by-beat with concept grounding	Essays and articles built over multiple sessions	3 (1/6– 7/5)
git-guardrails- claude-code	Installs a PreToolUse hook blocking destructive git commands (<code>push -- force</code> , <code>reset --hard</code> , <code>clean -f</code>)	Adding git safety rails on a machine without other guardrails	3 (2/9/1)
migrate-to- shoehorn	Replaces <code>as</code> type assertions in tests with <code>@total-typescript/shoehorn</code>	Exactly that TS test-typing migration, when you decide to do it	3 (1/9/3)
grill-with-docs / grill-me	One/two-line wrappers that invoke grilling (plus domain- modeling)	Manually triggering the interview primitive	2 (3/1– 2/1)
scaffold- exercises	Scaffolds exercise directories passing the ai-hero course's private linter	Only inside mattpocock's own course repos	2 (0/9/0)

Your recommended daily lineup

Situation	Reach for	Source
New feature / change	<code>/spec</code> → <code>/plan</code> → <code>/build auto</code>	agent-skills (installed)
Vague requirement	<code>interview-me</code> (auto-fires)	agent-skills (installed)
Production incident	<code>diagnosing-bugs</code> by name; <code>gstack investigate</code> for env archaeology	mattpocock (installed)
Exploring / feasibility research	<code>research</code> ; orientation file + doc index for repo mapping	mattpocock (installed)
Ops walkthrough (API signup, env, secrets)	<code>wizard</code>	mattpocock (installed)
Queue/pipeline API design	<code>api-and-interface-design</code>	agent-skills (promote)
Adding logging/metrics to outreach system	<code>observability-and-instrumentation</code>	agent-skills (promote)
Writing/editing harness skills	<code>writing-for-agents</code>	mattpocock (installed)
Visual change	sight loop (<code>/verify-ui</code> + <code>odiff</code>)	harness
Pre-merge	<code>/code-review</code> once	built-in
Live-system schema migration	<code>deprecation-and-migration</code> (from clone, break-glass)	agent-skills
Session end	<code>/wrap-session</code>	harness

Actions (pending your go-ahead)

1. Copy `api-and-interface-design` and `observability-and-instrumentation` from `.context/eval/agent-skills/skills/` into `~/.claude/skills/` (+ mirror to harness masters). Cost: ~2 description lines.

2. No changes to the mattpocock plugin — it's installed, current (1.2.3 = latest release), and costs ~650 tokens for its 11 visible skills, four of which score 6–7 for you.
3. Everything else in both repos: leave uninstalled; the clones in `.context/eval/` remain available for break-glass invocation.